RAZOR'S EDGE: THROUGHPUT OPTIMIZED DYNAMIC BATCHING WITH LATENCY OBJECTIVES

Anonymous Authors[1]

ABSTRACT

Serving systems for embedding, LLM, and other matrix-multiplication-dominated inference workloads rely on batching for efficient hardware utilization. We observe that batching efficiency exhibits a sharp input-size-dependent structure driven by the transition between memory-bound and compute-bound regimes: small inputs can be batched flexibly across heterogeneous sizes, while large inputs require near-uniformity, leading to a rapid collapse in batching efficiency. This produces a characteristic blade-like ("razor's edge") shape in the batch performance landscape.

We present the Razor's Edge batching scheduler, a practical framework that combines (i) dynamic-programming-based throughput optimization over sorted requests, (ii) production-oriented next-batch selection strategies (`FIFO`, `MINMAX`, and `GUARDED_BATCH_SIZE`), and (iii) startup-time-efficient model benchmarking that builds batch timing estimators from direct measurements on the same hardware where the model is deployed. A central novelty claim in this paper is this measurement-to-optimizer bridge: instead of relying on analytic proxy cost models, we benchmark the deployed model/hardware pair and feed those empirical timings directly into the DP cost table used for scheduling decisions. We also introduce a practical visualization method for quantifying batching efficiency improvements when expanding the allowed maximum batch size from (N-1) to (N), producing the characteristic "razor's edge" contour plots. The approach is designed for real-time online serving with queueing. Our claims are scoped to "ahead-of-time variable-size batching for encoder-style inference" evaluated in this paper, not to universal superiority across all serving stacks. We demonstrate the scheduler's efficacy through a 47% throughput increase on a CPU embedding workload (`jina-embeddings-v2-base-en`), a 26% throughput increase on a GPU embedding workload (`BAAI/bge-m3`), and controllable latency/throughput trade-offs across the final strategy set.

1 INTRODUCTION

Requests cannot be processed instantaneously at arrival in any real serving system. Under load, queueing is inevitable. A first-come, first-served single-request policy is simple, but it fails to exploit throughput gains available through batching. Simply batching the first `n` requests in a queue is common, but it is often not throughput-optimal. Sorting requests by size and then batching can create high latency for large requests that may not be selected quickly under sustained load.

This work addresses a common inference regime where workloads are dominated by batched matrix multiplication with variable input sizes (for example, variable token lengths in embedding or classification models). The scheduler pursues two goals simultaneously:

1. **Maximum throughput**: Sort and choose batch partitions that minimize total completion time for queued work.
2. **Latency objective**: choose near-term execution order with three production strategies:
   - (FIFO) choose the batch with the oldest input
   - (MINMAX) choose the batch with the highest prospective latency (waiting time + processing time)
   - (GUARDED_BATCH_SIZE) apply a MINMAX guardrail, then prefer larger eligible batches for throughput

In addition to runtime scheduling, we include startup benchmarking and estimator construction techniques that reduce calibration overhead while preserving scheduling quality. Concretely, the scheduler is parameterized by deployed-hardware benchmark data, so optimization is grounded in

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

measured batch inference timings rather than assumed proxy costs.

2 Related Work

This paper builds on two strands of prior work: (1) classical single-machine scheduling theory for completion-time style objectives, and (2) practical production batching systems used in modern ML inference.

2.1 Classical Scheduling Background

- Smith's foundational sequencing rule for weighted completion-time minimization gives the interchange-argument template used throughout modern scheduling analysis [1].
- For quadratic completion/lateness penalties, prior work studies single-machine objectives where squared terms increase the penalty for tail latency and variability [2], [3].

Our historical RMS ordering experiment is in this family: it used pairwise interchange logic under a squared-latency objective, adapted to grouped/batched online inference decisions. In final evaluation, we retain it only as a negative result for transparency.

2.2 Production ML Batching Systems

- **NVIDIA Triton dynamic batching** provides queue-delay windows and preferred batch sizes for online serving [4].
- **Infinity / batched integrations** expose practical dynamic batching controls for embedding and inference workloads [5], [6].
- **Hugging Face pipeline batching** documents practical throughput-oriented batch inference usage [7].

Unlike fixed-threshold accumulation approaches, Razor's Edge combines throughput-optimal partitioning (DP on sorted requests) with a second latency-aware selection pass from the final strategy set (`FIFO`, `MINMAX`, `GUARDED_BATCH_SIZE`).

2.3 Positioning Relative to Existing Batching Practice

Most production dynamic batchers expose preferred batch sizes, and simple first-ready policies [4]–[7]. These policies are robust and easy to deploy, but they typically do not solve an explicit global partitioning objective on the current queue state.

The key distinction of Razor's Edge is its two-stage decision process:

1. **Throughput-optimal contiguous partitioning** on size-sorted requests (DP over candidate cuts).
2. **Latency-aware first-batch selection** among DP-consistent candidates.

This makes Razor's Edge closer in spirit to objective-driven scheduling than to size-only batching heuristics, while remaining practical for online serving.

2.4 Scope of Novelty Claim

The contribution of this work is a systems-oriented synthesis with strictly bounded claims:

Algorithmic Synthesis: A practical sorted-queue batching model combining DP-based partitioning with a latency-aware ordering pass.

Deployment-Grounded Cost Modeling: The DP objective is instantiated with empirical timing values collected from the actual deployment target (model + runtime + hardware), rather than only synthetic or analytic estimates. This measurement-driven parameterization is treated as a first-class contribution of the framework.

Workload Specialization: This framework is explicitly designed for padding-heavy, fixed-window inference (e.g., BERT-style embeddings or classification) where the cost of a batch is dominated by its longest member.

Exclusion of Continuous Batching: This method is not intended for, nor applicable to, ”continuous batching” or ”iteration-level scheduling” (e.g., vLLM or TGI) used in causal LLM generation. It does not account for KV-cache management or mid-execution request insertion. The system uses ”ahead-of-time variable-size batching for encoder-style inference” where a batch is fully formed before being sent to the engine.

Empirical Gain: We validate that this synthesis yields operational gains in specific ”static” batching environments common in embedding and encoder-only transformer deployments.

What is **not** claimed:

- no proof of global optimality for the multi-group online or request ordering pass.
- no universal dominance over all dynamic batching policies, model families, or hardware environments.

What is **validated in this paper**:

- DP-based sorted contiguous partitioning and multiple first-batch heuristics (`FIFO`, `MINMAX`, `GUARDED_BATCH_SIZE`) can be integrated in an online executor with bounded per-decision overhead on the tested workload class.

- estimator calibration plus the scheduler improves measured throughput relative to the tested FIFO/fixed-cap baselines in Section 7 under the reported setups.

The empirical sections evaluate whether this synthesis yields operational gains in realistic serving settings under these scope conditions.

3 Problem Setting and Model Assumptions

Let

$$F([x_1, ..., x_n]) = [y_1, ..., y_n] \tag{1}$$

be a batch inference function executed on system S, returning one output per input.

3.1 Consistent Practical Utility

For an input (x_i), denote singleton output as (a), and output in different batching contexts as (y_i) or (d_i). Exact equality is not required; practical equivalence (e.g., for retrieval, ranking, or classification quality) is sufficient.

3.2 Consistent Timing

1. The execution time of a batch is independent of the internal ordering of requests within that batch.
2. Batch execution time is monotonically non-decreasing with both batch size and the largest input size in the batch.
3. If two batches have the same size and the same largest input size, they have the same execution time.
4. The largest element in a batch is the element with the highest singleton compute cost.

These 4 axioms imply that batching on sorted `x_i` is at least as fast as batching on unsorted partitions.

Proof (exchange argument sketch) Consider two internally ascending batches drawn from an optimal partition on unsorted inputs. We can always find two batches with at least two elements each:

$$B1 = [x_1, x_2, ...x_m] \tag{2}$$

$$B2 = [y_1, y_2, ...y_n] \tag{3}$$

with cross-overlap conditions `x_m` $>$ `y_1` and `y_n` $>$ `x_1`.

There are two cases:

1. If `x_m <= y_n`, construct:

$$B1' = [x_1, x_2, ...x_m - 1, y_1] \tag{4}$$

$$B2' = [x_m, y_2, ...y_n] \tag{5}$$

Then `time(B1') < time(B1)` while `time(B2') = time(B2)`. Repeating this local exchange step moves toward a partition that is consistent with sorted-input batching.

2. If `x_m > y_n`, construct:

$$B1" = [y_n, x_2, ...x_m] \tag{6}$$

$$B2" = [y_1, y_2, ...y_n - 1, x_1] \tag{7}$$

Then `time(B2") < time(B2)` while `time(B1") = time(B1)`. Repeating this exchange yields a partition equivalent to one obtainable from sorted inputs.

3.3 Applicability

The method is intended for **variable-size batched inference** workloads where a calibration phase can build a timing estimator over `(batch_size, max_input_size)` and where measured batch durations are stable enough for Section 3.2 assumptions to be useful. In this paper, that condition is tested on (i) a synthetic controlled workload and (ii) a `BAAI/bge-m3` GPU inference pipeline, and (iii) a `jinaai/jina-embeddings-v2-base-en` CPU inference pipeline under the reported hardware/runtime setup. Applicability to other PyTorch/TensorFlow/ONNX/OpenVINO deployments depends on whether their measured timing surfaces exhibit similar properties.

3.4 Limitations

The mathematical assumptions (Section 3.2) hold most firmly in environments where inputs are padded to the length of the longest element in the batch. In contrast, workloads utilizing FlashAttention-varlen or unpadding techniques have processing time that depends on the total sum of tokens rather than only the maximum length.

3.5 Notation and Timing Terms (used consistently below)

To avoid ambiguity, we use the following timing notation throughout:

- (a_i): request arrival timestamp.
- (s_i): service start timestamp for request (i).
- (c_i): completion timestamp for request (i).
- (q_i = s_i - a_i): queueing delay (waiting before service starts).
- ($\ell_i = c_i - a_i$): end-to-end latency.
- (D_k): processing duration of batch/group (k).

When referenced in historical sections, the RMS objective refers to minimizing $\sum_i \ell_i^2$ for the currently queued work considered by the scheduler.

4 Core Scheduling Method

Given a queue of requests, the scheduler runs four main steps:

1. We first order the requests based on their size.
2. We then partition them into batches to minimize total time using dynamic programming.
3. We do a single ordering pass using a latency objective.
4. We pick the first batch and update the expected start time of the next batch.

4.1 Batch creation by Dynamic Programming

The scheduler's primary objective is to partition a queue of N sorted requests into a set of batches $\mathcal{B} = \{b_1, b_2, \ldots, b_k\}$ such that the total processing time is minimized. Given that the hardware exhibits a non-linear relationship between batch size and input length, we formalize this as a shortest-path problem on a Directed Acyclic Graph (DAG).

Formal Definition Let $Q = \{r_1, r_2, \ldots, r_n\}$ be the set of requests in the queue, sorted by their input size s_i such that $s_1 \leq s_2 \leq \cdots \leq s_n$.

We define a cost function $C(b)$ that represents the expected execution time of a batch b. Based on our consistency axioms (Section 3.2), the execution time depends only on the number of elements in the batch $|b|$ and the size of the largest element $\max(s \in b)$. Since the queue is sorted, for any contiguous sub-sequence from index i to j, the cost is:

$$C(i, j) = \text{Estimator}[j - i + 1, s_j]$$

Recurrence Relation Let $T[j]$ be the minimum time required to process the first j requests. The optimal time for the prefix of length j is given by the recurrence:

$$T[j] = \min_{1 \leq k \leq M} \{T[j - k] + C(j - k + 1, j)\}$$

where M is the maximum allowable batch size.

We initialize $T[0] = 0$ and $T[j] = \infty$ for all $j > 0$. The complexity of this computation is $O(N \cdot M)$. Since M is typically small (< 256), this optimization is computationally efficient for real-time online serving.

Practical Implementation To ensure high-performance execution within the scheduling loop, the dynamic program is implemented using JIT-compiled functions. We pre-compute a 2D lookup table for $C(i, j)$, where the dimensions represent (batch size, input size). This allows for $O(1)$ cost lookups during the DP pass, minimizing the overhead added to the request lifecycle.

Implementation entry point: `get_batch_start_end_idx_and_duration` in `src/razors\_edge/optimal\_batching.py`.

Core internals:

- `_compiled_dynamic_batcher`
- `_get_slice_indexes_and_duration_minmax`
- `_get_slice_indexes_and_duration_fifo`
- `_get_slice_indexes_and_duration_guarded_batch_size`

4.2 MINMAX Optimizing Pass

This is the recommended final production strategy. It targets lower worst-case latency and is suitable for most systems. This strategy chooses the batch containing the request with the highest prospective latency (queueing delay + processing duration).

4.3 FIFO Optimizing Pass

This is a final production strategy for scenarios where fairness and oldest-first service are required. This strategy chooses the batch containing the oldest element. In our experiments, this strategy had slightly worse throughput and max latency compared to MINMAX.

4.4 GUARDED_BATCH_SIZE Optimizing Pass

This is the throughput-oriented production strategy used in the final scheduler set. The strategy first computes a queue-level mean MINMAX reference and admits only candidate batches whose prospective MINMAX score is at least that mean. Among admitted candidates, it selects the largest batch size; ties are broken by higher MINMAX score. This may lead to very high tail latencies and should only be used if throughput is prioritized over everything else like in batch processing regimes.

4.5 Negative Results for Picking the Next Batch (Not Included in Final Scheduler Guidance)

All of the below failed to show benefit

- Choose the batch to minimize RMSE latency (details below; it performed well only under low load)

- Choose the batch with the highest throughput (batch size / expected processing time)
- Choose the batch with the best efficiency (sum of time if each request was in largest batch size / time in current batch size)
- Choose the batch with the best performance gain (sum of time if each request was alone / time of current batch size)
- Choose the batch with the best (sum of wait times) / (expected processing time) (HRRN)

4.6 RMS Optimizing Pass (Negative Result)

We keep this section to document the RMS objective, derivation, and implementation we originally prioritized. In the latest benchmark validation cycle, RMS was not robust enough under sustained load and therefore is **not recommended** as a production scheduler in this paper's final guidance. The derivation is retained only for reproducibility and negative-result transparency.

Given request arrivals (time-ordered) `R1, R2, ...` at times `x0, x1, ...` such that `x_i < x_(i+1)`, with processing start times `y0, y1, ...` and output times `z0, z1, ...`, a system with minimum mean squared latency minimizes:

$$\sum_i (z_i - x_i)^2$$

over all i.

For a traditional one-by-one processor, $y_i = z_{i-1}$.

Two-request case Let `R1` and `R2` arrive while current request `R0` is processing.

Let:

- `R1` processing time be `D1`
- `R2` processing time be `D2`
- `R1` queueing delay be `W1`
- `R2` queueing delay be `W2`

$$\text{RMS_LATENCY}_1 = (W_1 + D_1)^2 + (W_2 + D_1 + D_2)^2$$

$$\text{RMS_LATENCY}_2 = (W_2 + D_2)^2 + (W_1 + D_1 + D_2)^2$$

If `RMS_LATENCY_1 < RMS_LATENCY_2`, pick `R1` before `R2`.

This simplifies to:

$$D_1\left(W_2 + D_2 + \frac{D_1}{2}\right) < D_2\left(W_1 + D_1 + \frac{D_2}{2}\right)$$

Grouped-requests case Consider groups `G1, G2, G3, ...`, processed one-by-one.

- Group durations: `D1, D2, D3, ...`
- Group end times: `T1 = D1, T2 = T1 + D2, ...` (positive, relative to when `G1` started)

Within group `i`, the arrival time of request `j` (negative relative to when `G1` started) is `Gi_Wj`.

To minimize RMS, order groups to minimize:

$$\sum_i \sum_j (G_{i,W_j} - T_i)^2$$

Which expands to

$$\sum_i \sum_j G_{i,W_j}^2 - 2\sum_i \left(T_i \sum_j G_{i,W_j}\right) + \sum_i |G_i| T_i^2$$

Ignoring the `sum(sum(Gi_Wj**2))` since it's constant, this simplifies to

$$\sum_i T_i |G_i| \left(T_i - 2\overline{G}_{i,W}\right)$$

This shows that exact per-request arrival terms are constant regardless of group ordering. The ordering-relevant term is the average arrival time within each group.

For 2 groups:

- Group `R1` has duration `D1`, size `N1`, average queueing delay `W1`
- Group `R2` has duration `D2`, size `N2`, average queueing delay `W2`

Define:

$$\text{RMS_LATENCY}_1 = N_1(W_1+D_1)^2 + N_2(W_2+D_1+D_2)^2$$

$$\text{RMS_LATENCY}_2 = N_2(W_2+D_2)^2 + N_1(W_1+D_1+D_2)^2$$

If `RMS_LATENCY_1 < RMS_LATENCY_2`, pick `R1` before `R2`.

This simplifies to:

$$N_2 D_1 \left(W_2 + D_2 + \frac{D_1}{2} \right) \; < \; N_1 D_2 \left(W_1 + D_1 + \frac{D_2}{2} \right)$$

Pick `R1` before `R2` when the inequality above holds.

In practice we use 64-bit integer nanosecond timing and limit batch size to 256 (`uint8`). For numerical safety, `log2((2**63 / (256 * 5)) ** 0.5) = 26.8247...`, so we conservatively use 26-bit scaled operands in the overflow-safe comparison path.

For more than 2 groups For more than two groups, this pairwise ordering property is not transitive, so a single greedy pass is not globally optimal in all cases. However, it is close to optimal under brute-force checks (`N`, `D`, and `T` from 1 to 10).

For three requests (`i`, `j`, `k`), we observe the optimal ordering starts with `i` in 99.995% of cases under two pairwise-consistency checks:

1. `i` should be processed first given only `i` and `j`, and `j` should be processed first given only `j` and `k` (47,066 / 997,002,000 suboptimal orderings).
2. `i` should be processed first given only `i` and `j`, and `i` should be processed first given only `i` and `k` (50,853 / 997,002,000 suboptimal orderings).

These edge cases are sufficiently rare for the intended real-time setting. Additional pre-sorting by required processing time before RMS comparison may recover small gains, but likely does not justify the extra scheduling overhead for online serving.

Intuitive example Prioritize `R2` for lower RMS latency when:

- `R1`: 1 request, 10 seconds processing, arrived 2 seconds ago
- `R2`: 2 requests, 1 second processing, arrived 1 second ago

$$2 \cdot 10 \cdot (2 + 2 + 10) \; < \; 1 \cdot 1 \cdot (4 + 20 + 1)$$

$$280 \; < \; 25$$

(false), so `R2` should be picked before `R1`.

If `W1` is treated as unknown, the crossover shows that if `R1` was already delayed by ~129.5s, processing `R1` first becomes optimal for RMS latency.

Implementation entry point: `get_batch_start_end_idx_and_duration` in `demos/scheduler\_tests/optimal\_batching.py`.

Core internals:

- `_compiled_bit_length`
- `_prospective_rms_latency_improvement`
- `_get_slice_indexes_and_duration_rms`

Mathematical mapping of these functions:

- `_compiled_bit_length(x)` computes

$$\mathrm{bitlen}(x) = \lfloor \log_2(x) \rfloor + 1, \quad x > 0$$

which is used to determine safe right-shift scales for 64-bit arithmetic.

- `_prospective_rms_latency_improvement(...)` compares two orderings of two candidate groups (`chosen` then `prospective` vs the reverse) with the same symbols and ordering semantics used in code:

  Let `c = chosen`, `p = prospective`, with `N` (group size), `D` (group duration), `W` (mean queueing time). The two-ordering squared-latency totals are:

$$L_{c \to p} = N_c (W_c + D_c)^2 + N_p (W_p + D_c + D_p)^2,$$

$$L_{p \to c} = N_p (W_p + D_p)^2 + N_c (W_c + D_p + D_c)^2$$

The decision in code is "prospective improves RMS latency" iff $L_{c \to p} > L_{p \to c}$. Expanding and cancelling common terms gives:

$$N_p D_c (2W_p + 2D_p + D_c) \; > \; N_c D_p (2W_c + 2D_c + D_p)$$

Divide both sides by (2) (positive, so direction is unchanged):

$$N_p D_c \left(W_p + D_p + \frac{D_c}{2} \right) \; > \; N_c D_p \left(W_c + D_c + \frac{D_p}{2} \right)$$

Reordering to match the exact comparator form in `optimal_batching.py`:

$$N_c D_p \left(W_c + D_c + \frac{D_p}{2} \right) \; < \; N_p D_c \left(W_p + D_p + \frac{D_c}{2} \right)$$

which is the same chosen-vs-prospective inequality shape implemented by `_prospective_rms_latency_improvement(...)`.

For numeric safety, the implementation right-shifts (>>) the time terms before multiplying, and uses integer half terms (`// 2`). These are monotone, positive scaling/rounding steps in the compared operands, so they preserve comparator direction for the scheduling decision while reducing overflow risk in 64-bit arithmetic.

- `_get_slice_indexes_and_duration_*(...)` reconstructs candidate batches along the optimal-throughput DP backpointer chain and selects

$$b^* = \arg\min_{b \in \mathcal{C}} \sum_{i \in b} (w_i + D_b)^2$$

in one pass via pairwise comparisons ($\mathcal{C}$ is the set of DP-derived contiguous candidates ending on the current suffix).

5 Benchmarking and Estimator Construction

A dynamic program requires a 2D array containing the expected duration to process a batch given its size and the size of the largest element, indexed as `arr[batch_size, max_input_size]`. Creating this array using exhaustive benchmarking would be prohibitively slow. Instead, for each batch size we measure selected points and extrapolate with a spline. For large enough `batch_size` and `max_input_size` we will stop seeing significant gains, where timings are likely to be scaled versions of the previous batch size: `arr[batch_size-1, max_input_size] * batch_size / (batch_size-1)` or lower. This typically indicates hardware saturation. To reduce startup cost, we detect this saturation point during benchmarking and reduce the maximum `max_input_size` for every `batch_size` where needed.

Function: `RazorsEdgeComputeTask.get_batch_timing_data` in `src/razors\_edge/razors\_edge\_compute\_task.py`.

The routine can be summarized as:

1. Generate initial input-size points

$$X_0 = \texttt{generate_benchmark_points}(s_{\min}, s_{\max}, \rho)$$

2. Measure timings for first two batch sizes (b_1, b_2):

$$T_{b_k}(x) \approx \text{bench}(b_k, x), \quad k \in \{1, 2\}$$

3. For each next batch size (b_m), adapt the next point set by a saturation heuristic:

$$X_m = \texttt{calculate_next_benchmark_points}(X_{m-2}, T_{m-2}, b_{m-2}, X$$

4. Pad late-stage points using scaled previous spline values:

$$\tilde{T}_m(x) = \alpha_m \cdot \hat{T}_{m-1}(x), \quad \alpha_m = \frac{b_m}{b_{m-1}}$$

for padding-only (x) values beyond the newly measured range.

5.1 CPU Benchmarking Strategy

CPU timing is possibly sensitive to garbage collection and inter-run state. The CPU protocol runs controlled sequences with GC and pause variations, then uses the **minimum observed time** as the hardware-limited best-case estimate.

Function: `model_test_pattern_cpu` in `src/razors\_edge/optimal\_benchmarking.py`.

Mathematically, if the run set is

$$\{t_1, \ldots, t_K\}$$

under different GC/sleep states, the estimator is:

$$\hat{t}_{\text{CPU}} = \min_{1 \le k \le K} t_k$$

which targets the lower-envelope compute cost under transient software noise.

5.2 GPU Benchmarking Strategy

GPU timings are sensitive to transient boost and warmup. The GPU protocol performs bounded repeated warm runs (time- and iteration-constrained) and uses the **median** measured value for stability.

Function: `model_test_pattern_gpu` in `src/razors\_edge/optimal\_benchmarking.py`.

If post-warmup measurements are $\{g_1, \ldots, g_K\}$, the estimator is:

$$\hat{t}_{\text{GPU}} = \text{median}(g_1, \ldots, g_K)$$

which is robust to occasional jitter spikes.

5.3 Adaptive Benchmark Point Reduction

To reduce startup cost, benchmark points are generated nonlinearly over token/input size and adapted using inter-batch timing ratio behavior (empirically sigmoid-like). If saturation is detected, the next benchmark range is reduced accordingly.

Functions:

- `get_points_ratio`
- `generate_benchmark_points`
- `calculate_next_benchmark_points`
- `get_benchmark_data_paddings`

(all in `src/razors\_edge/optimal\_benchmarking.py`).

Mathematical definitions:

- `get_points_ratio(min_tokens, max_tokens, max_points)`:

$$\rho = \frac{P_{\max} - 1}{\sqrt{s_{\max} - s_{\min}}} \tag{8}$$

- `generate_benchmark_points(...)` uses uniform spacing in $(\sqrt{s})$-space:

$$s_i = \left(\sqrt{s_{\min}} + \frac{i}{n-1}(\sqrt{s_{\max}} - \sqrt{s_{\min}})\right)^2, \quad i = 0, \ldots, n-1 \tag{9}$$

where

$$n = \max\left(\left\lfloor \sqrt{s_{\max} - s_{\min}}\,\rho + 1 \right\rfloor,\ \text{MINIMUM_SPLINE_POINTS}\right).$$

- `calculate_next_benchmark_points(...)` forms smoothed curves $(\hat{t}_1(s), \hat{t}_2(s))$ and ratio

$$r(s) = \frac{\hat{t}_2(s)/b_2}{\hat{t}_1(s)/b_1} \tag{10}$$

then enforces a monotone saturation envelope by reverse cumulative minimum and picks a new endpoint from sigmoid midpoint and saturation-threshold rules.

- `get_benchmark_data_paddings(...)` extends a shorter new curve by

$$t_{\text{pad}}(s) = \alpha \cdot \hat{t}_{\text{prev}}(s) \tag{11}$$

at padding points s, where α is the adjacent batch-size scaling ratio.

- `create_batch_timing_estimators(...)` builds

$$\text{Estimator}[b, s] \approx \hat{t}_b(s) \tag{12}$$

using spline interpolation for measured batch sizes and linear interpolation across neighboring benchmarked batch sizes:

$$\hat{t}_b(s) = (1-\lambda)\hat{t}_{b_{\text{lo}}}(s) + \lambda \hat{t}_{b_{\text{hi}}}(s), \quad \lambda = \frac{b - b_{\text{lo}}}{b_{\text{hi}} - b_{\text{lo}}}. \tag{13}$$

6 Systems Architecture

All ML workloads run in a single isolated process. Compute tasks are initialized inside that process. Model loading and optional heavyweight imports occur there, isolating runtime state from the caller process. Inputs are sent through a one-way pipe, and results are received on a separate pipe by a dedicated result-setting thread that resolves pending futures. When multiple task queues compete, the queue with the earliest pending request key is selected first. Backpressure and concurrency are controlled by thread pools and semaphores. In the current implementation, unrecoverable failures raise `SystemExit` so the host application can restart cleanly.

Primary class: `ComputeExecutor` in `src/executor/process\_manager.py`.

6.1 Process and Threading Design

Execution is isolated in a spawned process with:

- bounded async/sync admission,
- dedicated send/receive channels,
- result-setting loop,
- periodic inter-process time synchronization,
- fair queue selection by earliest operation id.

Key methods:

- `async_compute_fn`, `sync_compute_fn`
- `set_result_loop`, `set_time_loop`
- internal `_InternalProcess.accumulate_data`, `choose_fair_queue`, `post_process_and_send_result`

Mathematical interpretation:

- `choose_fair_queue` performs earliest-key selection:

$$q^* = \arg\min_q (o_q,\ \tau_q) \tag{14}$$

where (o_q, τ_q) is the first pending `(operation_id, queue_time)` in queue (q).

- `set_time_loop` / `_InternalProcess.get_time_loop` maintain clock alignment by repeatedly updating

$$\Delta = t_{\text{internal}} - t_{\text{external}} \tag{15}$$

and using

$$\tau_{\text{aligned}} = \tau_{\text{recv}} + \Delta$$

for queue timestamps.

- `post_process_and_send_result` is an order-preserving map over batch ids:

$$\{(id_i, y_i)\}_{i=1}^{m}, \quad y_i = \text{postprocess}(\text{model}(x_{1:m}))_i \tag{16}$$

with one emitted result per original request id.

6.2 Compute Task Contract

The base plugin interface defines preprocessing, batching acceptance, model call, and postprocessing hooks.

Base class: `BaseBatchedComputeTask` in `src/executor/base\_batched\_compute\_task.py`.

For the default base implementation, batching is a maximal prefix under acceptance predicate (A):

$$b^* = (x_1, \ldots, x_k), \quad k = \max\{j : A((x_1, \ldots, x_{j-1}), x_j) = \text{True}\} \tag{17}$$

with default $A = \text{False}$, giving $k = 1$ (single-item batches).

6.3 Razor's Edge Compute Task Contract

This interface extends the base plugin and adds hooks for model loading, preprocessing, postprocessing, benchmark generation, and batch creation. The benchmark generation is cached by the model loading function.

Base class: `RazorsEdgeComputeTask` in `src/razors_edge/razors_edge_compute_task.py`.

At runtime, batch selection is a composition:

$$\text{inputs} \xrightarrow{\text{preprocess}} (u_i, s_i) \xrightarrow{\text{sort by } s_i} \xrightarrow{\text{DP+Latency}} [i:j] \xrightarrow{\text{create_batch}} B \xrightarrow{\text{model}} \hat{y} \xrightarrow{\text{post}} \tag{18}$$

and expected start-time tracking updates as

$$t_{\text{expected}} \leftarrow \max(t_{\text{expected}}, t_{\text{now}}) + D_{[i:j)}. \tag{19}$$

7 Results

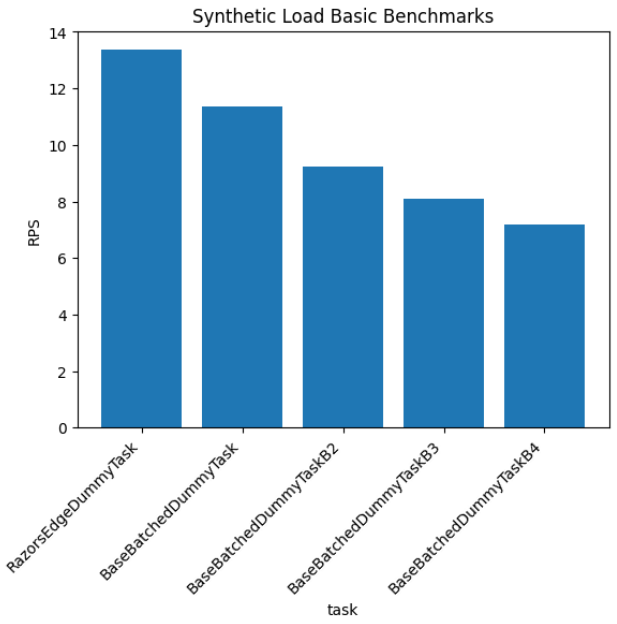
We evaluate Razor's Edge on both synthetic and real inference workloads to study batching behavior under controlled and realistic conditions.

Unless otherwise noted, each reported throughput statistic is based on 5 independent notebook runs per setting. We report means and sample standard deviations. Plotted curves/contours use representative (lower-noise) runs for visual clarity.

For the synthetic task, Razor's Edge achieved a throughput of 13.3 RPS (17% higher than the baseline), with a maximum allowed batch size of 8. For the same synthetic task, the baseline strategy achieved a peak throughput of 11.4 RPS, with best performance when the batch size was limited to 1. In this setting, simple batching did not improve throughput.

Figure 1: Synthetic load benchmark comparison.

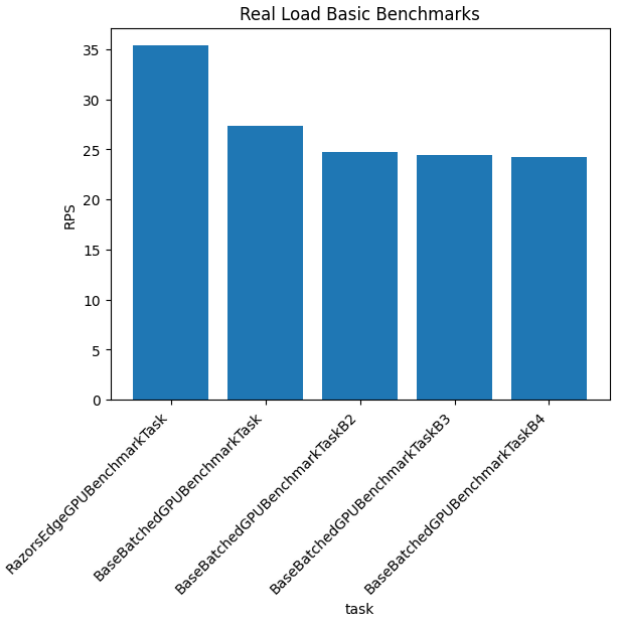
Notebook source: `demos/synthetic/dummy_performance_comparison_basic.ipynb`.



For the `BAAI/bge-m3` GPU workload, Razor's Edge achieved a throughput of 35.3 RPS (25% higher than the baseline), with a maximum allowed batch size of 16. For the real model workload, the baseline strategy achieved a throughput of 28.2 RPS, with best performance when the batch size was limited to 1. In this setting, simple batching did not improve throughput.

Figure 2: `BAAI/bge-m3` GPU workload benchmark comparison.

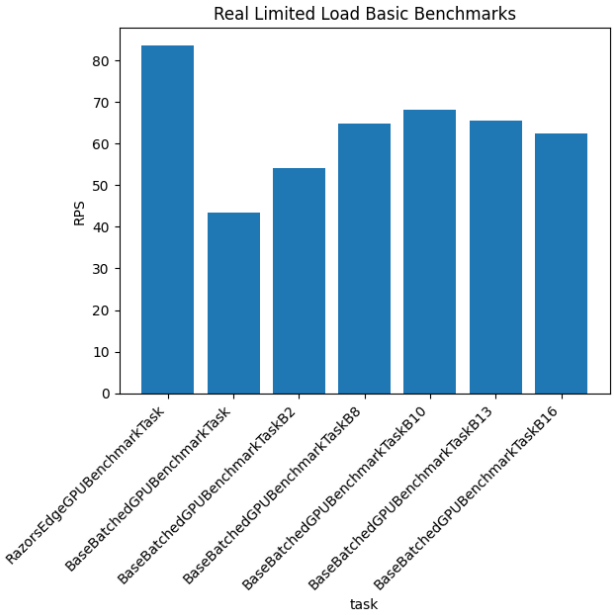
Notebook source: `demos/real/gpu_benchmark_performance_comparison_basic.ipynb`.



For the `BAAI/bge-m3` GPU workload with limited token input, Razor's Edge achieved a throughput of 84.6 RPS (26% higher than the baseline with batch size = 10), with a maximum allowed batch size of 16. For this limited-token workload, the baseline strategy achieved a throughput of 67.4 RPS, with best performance at batch size = 10. Baseline batching improved throughput relative to the non-batched baseline of 41.4 RPS (39% lower than the baseline with batch size = 10).

Figure 3: `BAAI/bge-m3` GPU workload benchmark comparison with limited tokens.

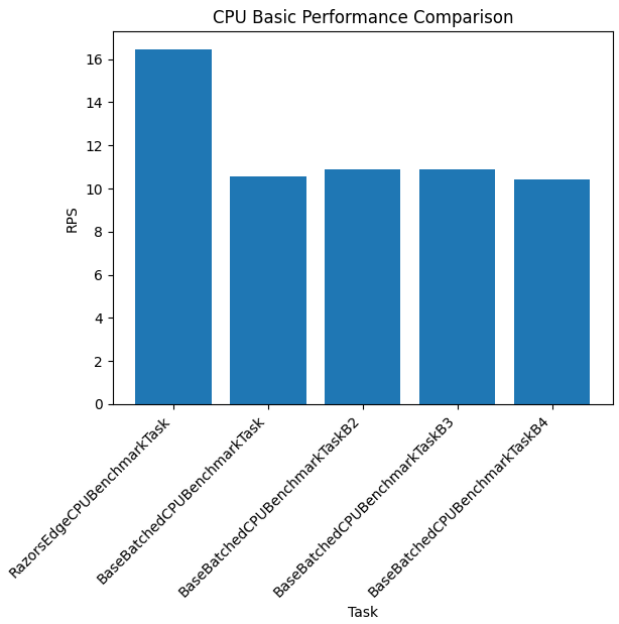
Notebook source: `demos/real/gpu_benchmark_performance_comparison_basic_limited.ipynb`.



For the `jinaai/jina-embeddings-v2-base-en` CPU workload, Razor's Edge achieved a throughput of 16.2 RPS (47% higher than the baseline with batch 2 and no batch), with a maximum allowed batch size of 6. For this workload, the baseline strategy achieved a throughput of 11.0 RPS, with best performance at batch size 2. Baseline batching marginally improved throughput relative to the non-batched baseline of 10.5 RPS (5% lower than the baseline with batch size = 2).

Figure 4: Real `jinaai/jina-embeddings-v2-base-en` CPU benchmark comparison.

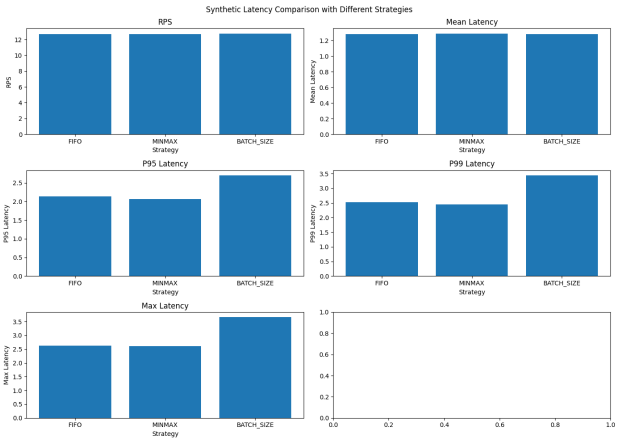
Notebook source: `demos/cpu/razors_edge_cpu_benchmark_task.py`.



For the synthetic workload, we compare the three outlined latency objectives.

Figure 5: Synthetic latency comparison.

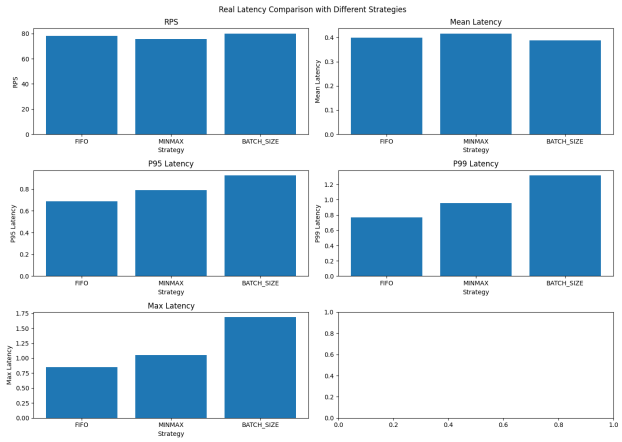
Notebook source: `demos/synthetic/dummy_latency_comparison.ipynb`.



For the `BAAI/bge-m3` GPU workload with limited tokens, we compare the three outlined latency objectives.

Figure 6: Real `BAAI/bge-m3` GPU latency comparison.

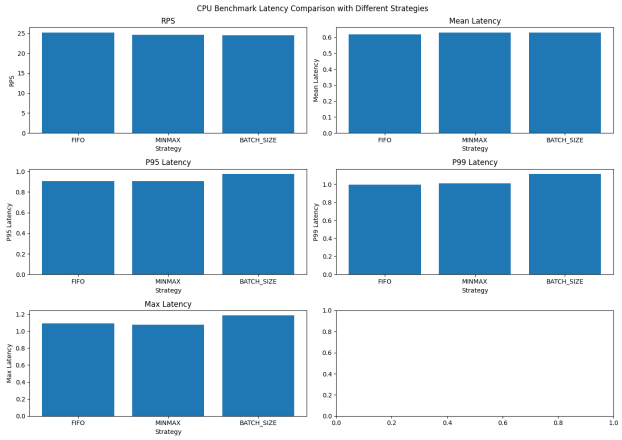
Notebook source: `demos/real/gpu_benchmark_limited_latency_comparison.ipynb`.



For the `jinaai/jina-embeddings-v2-base-en` CPU workload, we compare the three outlined latency objectives.

Figure 7: Real `jinaai/jina-embeddings-v2-base-en` CPU latency comparison.

Notebook source: `demos/cpu/cpu_benchmark_latency_comparison.ipynb`.



7.1 Baseline Policy Definition

The baseline in Section 7 is the `BaseBatchedComputeTask` policy run under `ComputeExecutor`, with behavior as implemented:

- **FIFO oldest-first selection (no reordering/sorting):** each per-task queue is a Python `dict` keyed by `(operation_id, queue_time)` and consumed from iteration order; `choose_fair_queue` picks the queue whose first key is globally smallest, and `get_batch_ids_list_and_batch` consumes from the first enqueued item onward. This preserves arrival order and does not apply size-based sorting. Implementation points:
  - `ComputeExecutor._InternalProcess.`

accumulate_data
  - ComputeExecutor._InternalProcess.choose_fair_queue
  - BaseBatchedComputeTask.get_batch_ids_list_and_batch
- **Immediate dispatch on idle:** when all queues are empty, accumulate_data blocks until at least one request arrives; the loop then immediately selects a fair queue, forms a batch, removes ids, and submits execution without any delay.
  Implementation points:
  - ComputeExecutor._InternalProcess.__init__ main loop
  - ComputeExecutor._InternalProcess.accumulate_data
- **Append until max batch size n:** baseline task variants implement _accept_in_batch as len(current_batch) < self.max_batch_size, so batching is a maximal prefix up to fixed cap n.
  Implementation points: demos/synthetic/base_batched_dummy_task.py and demos/real/base_batched_gpu_benchmark_task.py (both define max_batch_size and this predicate), plus variant classes that set n.

Exact baseline hyperparameter grids (n) used in Section 7 comparisons

- **Synthetic basic benchmark grid:** n in {1, 2, 3, 4} (via BaseBatchedDummyTask, BaseBatchedDummyTaskB2, BaseBatchedDummyTaskB3, BaseBatchedDummyTaskB4).
- **Real basic benchmark grid:** n in {1, 2, 3, 4} (via BaseBatchedGPUBenchmarkTask, BaseBatchedGPUBenchmarkTaskB2, BaseBatchedGPUBenchmarkTaskB3, BaseBatchedGPUBenchmarkTaskB4).
- **Real limited-token benchmark grid:** n in {1, 2, 8, 10, 13, 16}.

Method used to choose reported baseline setting

For each workload, we benchmark each candidate n over 5 runs and report the mean throughput (RPS) of 5 runs. This selected setting is the one reported in Section 7 for that workload.

7.2 Data

For throughput, every run is reported.

For latency, we report the mean across 5 runs (measured in the same notebooks).

Throughput data

Latency strategy tables

Synthetic workload

BAAI/bge-m3 GPU Workload (limited tokens)

jinaai/jina-embeddings-v2-base-en CPU workload

7.3 Estimator-Array Replay Validation

We also replayed key Section 7 throughput claims using only the saved estimator arrays and a saturated simulation harness.

The estimator arrays were generated using:

- demos/scheduler_tests/save_estimators_for_simulation.ipynb

Replay estimator files:

- demos/scheduler_tests/estimator_arrays/est_store_synthetic.txt
- demos/scheduler_tests/estimator_arrays/est_store_bge_m3.txt
- demos/scheduler_tests/estimator_arrays/est_store_jina.txt

Method:

- 5 seeds ([42, 43, 44, 45, 46]), 20,000 requests per seed, queue cap = 16.
- Baseline is FIFO arrival-order fixed-cap batching, sweeping all supported n values and selecting best mean-RPS baseline.
- Razor's Edge replay compares final strategies (FIFO, MINMAX, BATCH_SIZE) on sorted+DP candidate chains.

These replay values are reported raw from estimator-array simulation and are intended as a consistency/sensitivity check alongside the end-to-end runtime notebook measurements.

7.4 Ablative Study (Estimator Replay)

We isolate each mechanism in the scheduling stack with the following variants:

Workload	Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Mean	Std
Synthetic Load	Razor's Edge (best final strategy)	13.3	13.3	13.2	13.3	13.2	13.3	0.05
	No batching	11.4	11.3	11.4	11.4	11.4	11.4	0.04
BAAI/bge-m3 GPU	Razor's Edge (best final strategy)	35.8	35.5	36.3	34.4	34.4	35.3	0.76
	No batching	27.9	28.1	27.6	29.0	28.4	28.2	0.48
BAAI/bge-m3 GPU (Limited tokens)	Razor's Edge (best final strategy)	85.1	82.8	83.3	86.0	85.9	84.6	1.33
	Baseline (BS=10)	68.1	69.0	68.3	66.0	65.5	67.4	1.37
	No batching	42.0	37.9	45.5	41.1	40.6	41.4	2.46
jinaai/jina-embeddings-v2-base-en CPU	Razor's Edge (best final strategy)	16.3	16.1	16.4	16.2	16.2	16.2	0.10
	Baseline (BS=2)	11.0	10.9	11.0	11.0	10.9	11.0	0.05
	No batching	10.5	10.8	10.5	10.2	10.7	10.5	0.21

Strategy	RPS	Mean Latency	P95 Latency	P99 Latency	Max Latency
FIFO	12.710969	1.282433	2.132870	2.521277	2.626158
MINMAX	12.702064	1.289926	2.069032	2.456606	2.616079
GUARDED_BATCH_SIZE	12.766899	1.282302	2.700126	3.443307	3.660290

- **A**: baseline fixed-cap FIFO (unsorted queue).
- **B**: sort-only fixed-cap FIFO (same `n` as variant A).
- **C**: sort + DP + FIFO objective.
- **D**: sort + DP + MINMAX objective.
- **E**: sort + DP + BATCH_SIZE objective.
- **F**: sort + greedy batch-construction + MINMAX objective.

Incremental mean-RPS deltas:

- Synthetic: `B-A = 0.000`, `D-B = 1.712`, `C-D = -0.006`, `E-D = 0.034`.
- `BAAI/bge-m3`: `B-A = 0.037`, `D-B = 9.249`, `C-D = 0.317`, `E-D = -0.572`.
- `jinaai/jina-embeddings-v2-base-en`: `B-A = 0.000`, `D-B = 0.882`, `C-D = -0.000`, `E-D = 0.003`.
- Greedy(MINMAX)-vs-DP(MINMAX) (`F-D`): Synthetic `-0.108`, `BAAI/bge-m3` `-5.805`, `jinaai/jina-embeddings-v2-base-en` `-0.250`.

This mechanism-isolation replay is included as a simulation-only complement to the end-to-end runtime comparisons in Section 7.

7.5 Threats to Validity for Replay Results

The estimator-array replay results in Section 7.2.1-7.2.2 should be interpreted with the following constraints:

1. **Simulation layer vs runtime system layer.**
   Replay uses calibrated estimator arrays and scheduler logic, but does not model framework/runtime effects such as Python overhead variance, CUDA stream contention, allocator behavior, and OS-level scheduling jitter.

2. **Closed-loop saturated queueing only.**
   The replay fills queues to capacity each scheduler loop. This is useful for stress testing batching behavior but does not cover open-loop arrival processes, burstiness, or non-stationary traffic.

3. **Sorting-sensitive latency artifacts in ablation.**
   The sort-only ablation variant changes queue order, which can produce latency changes that reflect reordering policy rather than pure compute-efficiency gains.

4. **Baseline family scope.**
   Replay baselines are fixed-cap FIFO families. These are valid operational baselines for this repository but are not exhaustive across all production serving policies.

Accordingly, Section 7.2 is presented as a reproducible sensitivity/consistency analysis, while end-to-end notebook measurements remain the primary empirical evidence for absolute throughput and latency claims.

7.6 Hardware and Software

Hardware and OS Experiments were conducted on an ASUS ROG Zephyrus Duo 16 laptop.

- CPU: AMD Ryzen 9 6900HX (16 threads) (power limits: SPL 20W, sPPT 20W, fPPT 25W)
- GPU: NVIDIA RTX 3080 Ti Laptop GPU (835 MHz fixed max clock, 185 MHz core OC, 210 MHz memory

Strategy	RPS	Mean Latency	P95 Latency	P99 Latency	Max Latency
FIFO	25.186837	0.616967	0.904827	0.997882	1.090114
MINMAX	24.636417	0.630478	0.904499	1.012017	1.072519
GUARDED_BATCH_SIZE	24.498417	0.628991	0.974383	1.114459	1.183429

Strategy	RPS	Mean Latency	P95 Latency	P99 Latency	Max Latency
FIFO	29.325394	3.416181	6.473206	6.775366	6.814931
MINMAX	28.298121	3.680978	6.969277	7.067753	7.068030
GUARDED_BATCH_SIZE	30.585454	3.231145	6.223230	6.481141	6.545033

OC, 5 W dynamic boost, 80 degreeC temperature target)

- OS: Windows 24H2 IoT with AtlasOS modifications
- Cooling: maximum fan speeds; no thermal throttling observed

The most stable practical environment was used, with no background application load and Python run at high process priority (using `psutil`) to improve timing stability. Graphs/figures shown are least-noisy representative runs for visual readability, while all throughput values reported in text are means across 5 runs per setting.

Software Python 3.14.3 (`pip list` output):

```
Package                    Version
-------------------------- ------------
accelerate                 1.13.0
annotated-doc              0.0.4
anyio                      4.12.1
argon2-cffi                25.1.0
argon2-cffi-bindings       25.1.0
arrow                      1.4.0
asttokens                  3.0.1
async-lru                  2.3.0
attrs                      26.1.0
babel                      2.18.0
beautifulsoup4             4.14.3
bleach                     6.3.0
certifi                    2026.2.25
cffi                       2.0.0
charset-normalizer         3.4.6
click                      8.3.1
colorama                   0.4.6
comm                       0.2.3
contourpy                  1.3.3
coverage                   7.13.5
cycler                     0.12.1
debugpy                    1.8.20
decorator                  5.2.1
defusedxml                 0.7.1
executing                  2.2.1
fastjsonschema             2.21.2
filelock                   3.25.2
flatbuffers                25.12.19
fonttools                  4.62.1
fqdn                       1.5.1
fsspec                     2026.2.0
h11                        0.16.0
hf-xet                     1.4.2
httpcore                   1.0.9
httpx                      0.28.1
huggingface_hub            0.36.2
idna                       3.11
ipykernel                  7.2.0
ipython                    9.11.0
ipython_pygments_lexers    1.1.1
ipywidgets                 8.1.8
isoduration                20.11.0
jedi                       0.19.2
Jinja2                     3.1.6
joblib                     1.5.3
json5                      0.13.0
jsonpointer                3.0.0
jsonschema                 4.26.0
jsonschema-specifications  2025.9.1
jupyter                    1.1.1
jupyter_client             8.8.0
jupyter-console            6.6.3
jupyter_core               5.9.1
jupyter-events             0.12.0
jupyter-lsp                2.3.0
jupyter_server             2.17.0
jupyter_server_terminals   0.5.4
jupyterlab                 4.5.6
jupyterlab_pygments        0.3.0
jupyterlab_server          2.28.0
jupyterlab_widgets         3.0.16
kiwisolver                 1.5.0
lark                       1.3.1
llvmlite                   0.46.0
markdown-it-py             4.0.0
MarkupSafe                 3.0.3
```

Workload	Best baseline RPS	Best Razor's Edge strategy	Best Razor's
Synthetic	11.348	BATCH_SIZE	
BAAI/bge-m3 (GPU estimator replay)	27.887	FIFO	
jinaai/jina-embeddings-v2-base-en (CPU estimator replay)	3.488	BATCH_SIZE	

Workload	Fixed n used for A/B	A mean RPS	B mean RPS	C mean RPS	D mean
Synthetic	1	11.348	11.348	13.054	13
BAAI/bge-m3 replay	3	27.887	27.924	37.490	37
jinaai/jina-embeddings-v2-base-en replay	1	3.488	3.488	4.370	4

matplotlib	3.10.8
matplotlib-inline	0.2.1
mdurl	0.1.2
mistune	3.2.0
ml_dtypes	0.5.4
mpmath	1.3.0
nbclient	0.10.4
nbconvert	7.17.0
nbformat	5.10.4
nest-asyncio	1.6.0
networkx	3.6.1
notebook	7.5.5
notebook_shim	0.2.4
numba	0.64.0
numpy	2.4.3
onnx	1.20.1
onnxruntime	1.24.4
optimum	2.1.0
optimum-onnx	0.1.0
packaging	25.0
pandocfilters	1.5.1
parso	0.8.6
pillow	12.1.1
pip	26.0.1
platformdirs	4.9.4
prometheus_client	0.24.1
prompt_toolkit	3.0.52
protobuf	7.34.1
psutil	7.2.2
pure_eval	0.2.3
pycparser	3.0
Pygments	2.19.2
pyparsing	3.3.2
python-dateutil	2.9.0.post0
python-json-logger	4.0.0
pywinpty	3.0.3
PyYAML	6.0.3
pyzmq	27.1.0
referencing	0.37.0
regex	2026.2.28
requests	2.32.5
rfc3339-validator	0.1.4
rfc3986-validator	0.1.1
rfc3987-syntax	1.1.0
rich	14.3.3
rpds-py	0.30.0
safetensors	0.7.0
scikit-learn	1.8.0
scipy	1.17.1
Send2Trash	2.1.0
setuptools	80.10.2
shellingham	1.5.4
six	1.17.0
soupsieve	2.8.3
stack-data	0.6.3
sympy	1.14.0
terminado	0.18.1
threadpoolctl	3.6.0
tinycss2	1.4.0
tokenizers	0.22.2
torch	2.10.0+cu126
tornado	6.5.5
tqdm	4.67.3
traitlets	5.14.3
transformers	4.57.6
typer	0.24.1
typing_extensions	4.15.0
tzdata	2025.3
uri-template	1.3.0
urllib3	2.6.3
wcwidth	0.6.0
webcolors	25.10.0
webencodings	0.5.1
websocket-client	1.9.0
wheel	0.46.3
widgetsnbextension	4.0.15

7.7 Workloads

Synthetic workload: We construct a dummy model where processing time is a function of input size and batch size. This isolates batching effects and enables clear visualization of performance structure. The synthetic

workload uses an accurate delay based on batch input size. Final strategy comparisons use FIFO, MINMAX, and GUARDED_BATCH_SIZE. Throughput numbers use MINMAX (recommended default). Real GPU workload: We evaluate a batched inference pipeline using the `BAAI/bge-m3` model on GPU. CPU workload: We evaluate the shape of the batching performance contours of `jinaai/jina-embeddings-v2-base-en` on CPU.

Requests were created using uniform-random characters with varying lengths:

- 1 to 1000 characters for the synthetic task
- 1 to 1300 chars (corresponding to 1 to 1000 tokens) for the real task `BAAI/bge-m3`
- 1 to 500 chars (corresponding to 1 to 400 tokens) for the limited real task `BAAI/bge-m3`
- 1 to 200 chars for the CPU task `jinaai/jina-embeddings-v2-base-en`

7.8 Throughput under Increasing Load

We next evaluate system throughput as a function of load by varying the number of concurrent requests.

The comparison figures in this section show Razor's Edge against the baseline dynamic batching strategy. The baseline exhibits approximately constant throughput as load increases, reflecting its reliance on fixed-size aggregation without improved batching quality. In contrast, Razor's Edge shows increasing throughput with load up to saturation.

Graphs

Synthetic **Figure 8:** Baseline scheduler throughput vs. parallelism (synthetic workload).

Notebook source: `demos/synthetic/dummy_performance_comparison.ipynb`.

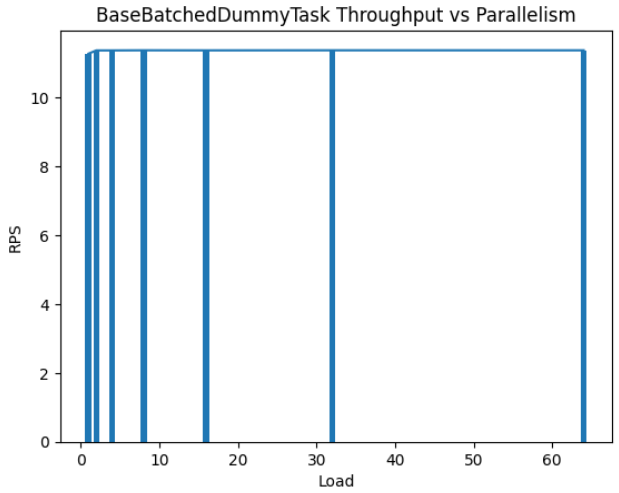


Figure 9: Razor's Edge scheduler throughput vs. parallelism (synthetic workload).

Notebook source: `demos/synthetic/dummy_performance_comparison.ipynb`.

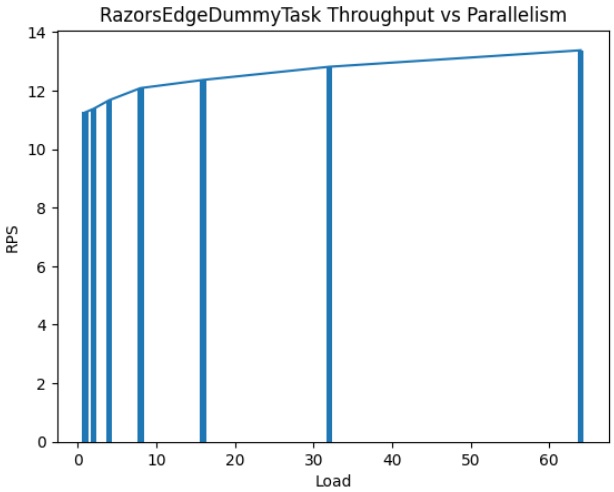
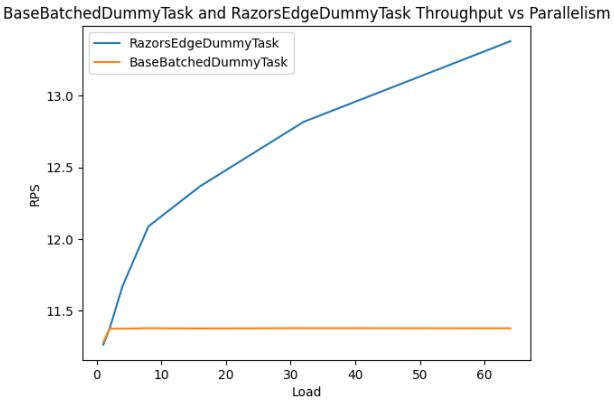


Figure 10: Direct throughput comparison of baseline and Razor's Edge (synthetic workload).

Notebook source: `demos/synthetic/dummy_performance_comparison.ipynb`.



BAAI/bge-m3 GPU workload **Figure 11:** Baseline scheduler throughput vs. parallelism (`BAAI/bge-m3` GPU workload).

Notebook source: `demos/real/gpu_benchmark_performance_comparison.ipynb`.

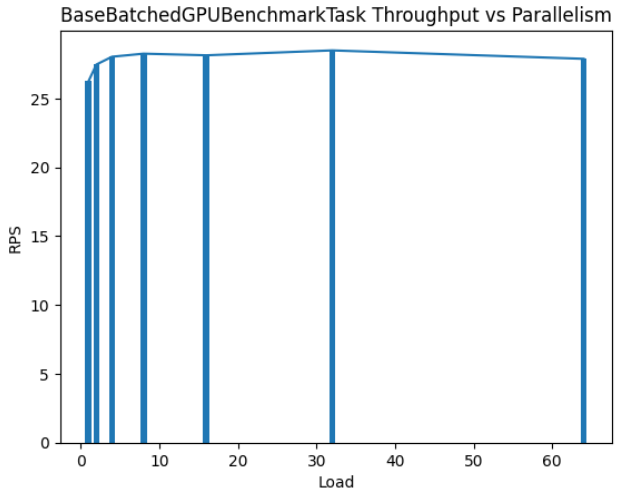


Figure 12: Razor's Edge scheduler throughput vs. parallelism (`BAAI/bge-m3` GPU workload).

Notebook source: `demos/real/gpu_benchmark_performance_comparison.ipynb`.

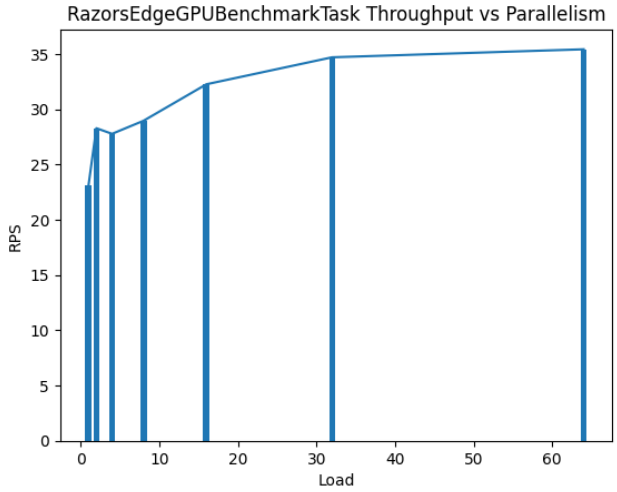
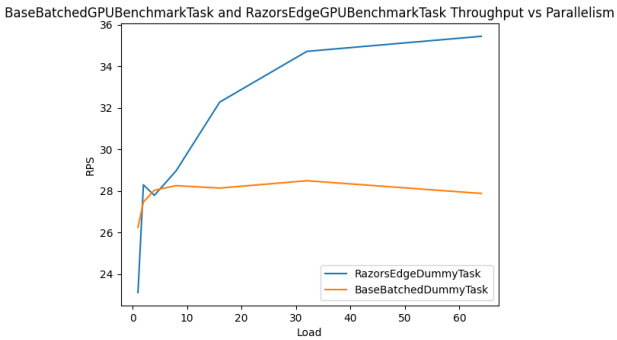


Figure 13: Direct throughput comparison of baseline and Razor's Edge (`BAAI/bge-m3` GPU workload).

Notebook source: `demos/real/gpu_benchmark_performance_comparison.ipynb`.



jinaai/jina-embeddings-v2-base-en CPU workload **Figure 14:** Baseline scheduler throughput vs. parallelism (`jinaai/jina-embeddings-v2-base-en` CPU workload).

Notebook source: `demos/cpu/cpu\performance\comparison.ipynb`.

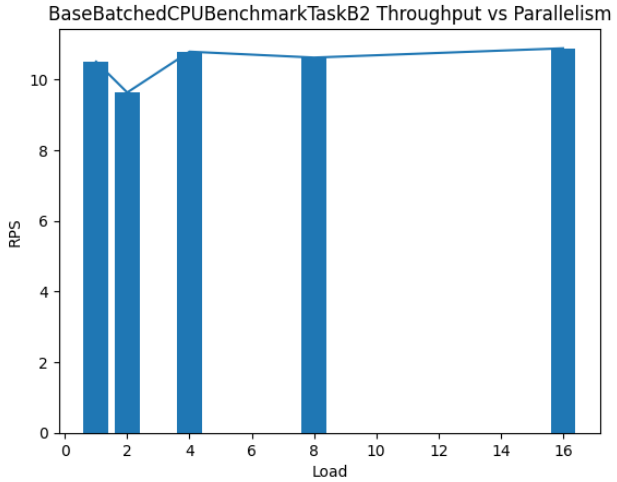


Figure 15: Razor's Edge scheduler throughput vs. parallelism (`jinaai/jina-embeddings-v2-base-en` CPU workload).

Notebook source: `demos/cpu/cpu_performance_comparison.ipynb`.

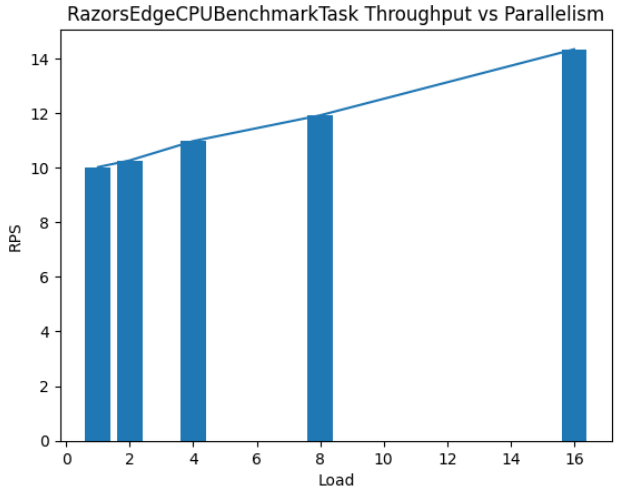
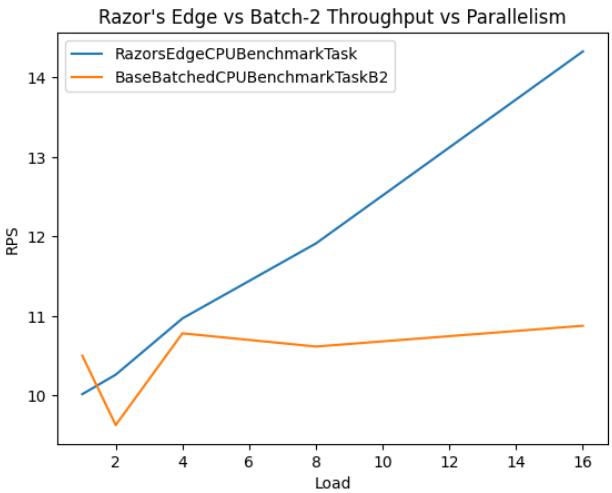


Figure 16: Direct throughput comparison of baseline and Razor's Edge (`jinaai/jina-embeddings-v2-base-en` CPU workload).

Notebook source: `demos/cpu/cpu_performance_comparison.ipynb`.



Reasoning This effect arises because larger queue sizes enable better sorting of input sizes, allowing the scheduler to construct more efficient batches. As concurrency increases, the dynamic program has greater flexibility to select near-optimal partitions, improving overall hardware utilization.

We observe this trend clearly in both synthetic and real workloads. These results demonstrate that Razor's Edge can exploit queue depth to improve batching efficiency, partially offsetting increased-load effects.

7.9 Razor's Edge Structure in Batching Efficiency

We next describe a visualization method used in this work to measure batching efficiency structure.

For a fixed target size (N), define a queue (Q_{x,y,N}) of exactly (N) requests with input sizes drawn uniformly from ([x,y]) (inclusive). For each ((x,y)) pair:

1. Compute **forced-(N)** time, where all (N) requests are forced into one batch of size (N):

$$T_N^{\text{forced}}(x, y) \tag{20}$$

2. Compute **dynamic-((N-1))** best-case time using the same DP partitioning method introduced earlier, but with maximum allowed batch size (N-1):

$$T_{N-1}^{\text{dynamic}}(x, y) \tag{21}$$

3. Form the efficiency ratio:

$$R_N(x, y) = \frac{T_N^{\text{forced}}(x, y)}{T_{N-1}^{\text{dynamic}}(x, y)} \tag{22}$$

We then plot a grayscale heatmap over ((x,y)), with (x) on one axis and (y) on the other, using (R_N(x,y)) as intensity. We additionally draw contour lines at:

$$R_N(x, y) \in \{0.85,\ 0.9,\ 0.95,\ 1.0\} \tag{23}$$

to visualize transitions in batching efficiency.

Interpretation:

- ($R_N(x, y) \ll 1$): forcing batch size N is substantially more efficient than the best dynamic strategy restricted to max size (N-1).
- ($R_N(x, y) \approx 1$): little to no efficiency gain from allowing (N) instead of (N-1).
- The contour transition region forms the observed "razor's edge" shape.

On the synthetic workload, these contours show that smaller inputs permit broader size mixing while larger inputs require tighter within-batch uniformity. In this setup, this trend is consistent with a transition from memory-dominated to compute-dominated behavior.

In the real `BAAI/bge-m3` GPU workload, we observe a qualitatively similar contour trend with higher measurement noise. In the real `jinaai/jina-embeddings-v2-base-en` CPU workload, we again observe a similar contour structure, with a narrower high-efficiency region consistent with stronger compute-bound penalties for mismatch.

These measured contour structures support applicability to the evaluated workload class (variable-size batched inference with calibrated estimators). We do not claim the same structure for all untested models and hardware.

Figure 17: Improvement from allowing variable batch sizes (synthetic task performance contours).

Notebook source: `demos/synthetic/razors_edge_dummy_graphs.ipynb`.

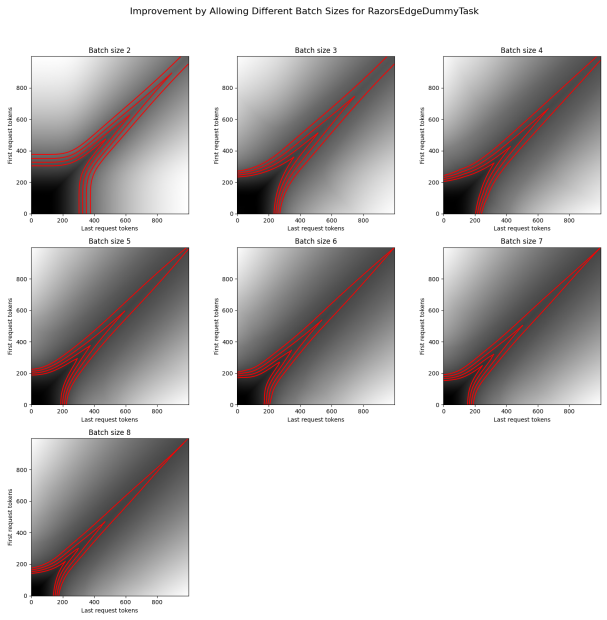


Figure 18: Improvement from allowing variable batch sizes (real-model GPU performance contours).

Notebook source: `demos/real/razors\edge\gpu\benchmark\graphs.ipynb`.

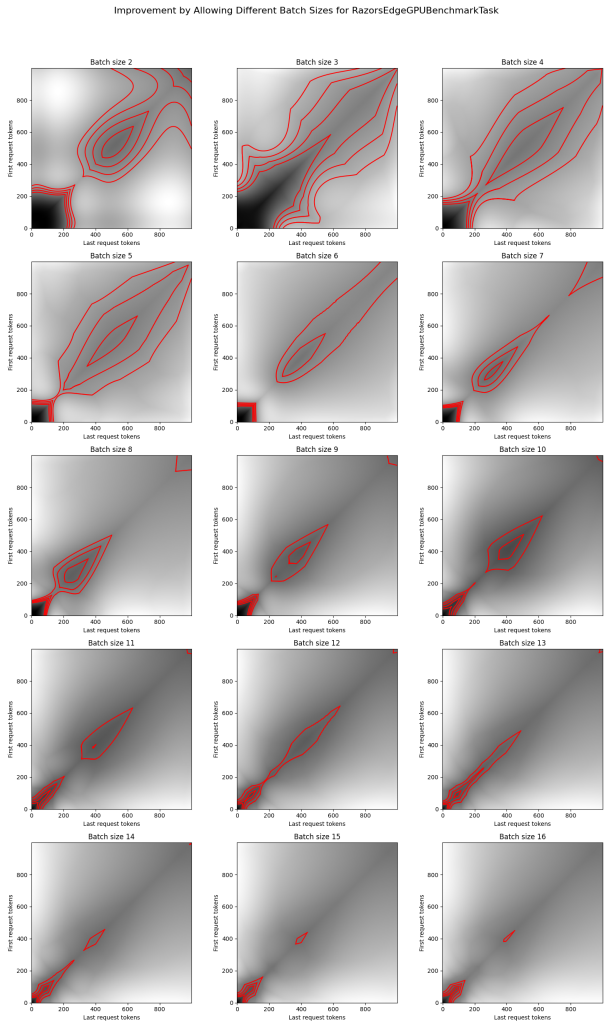
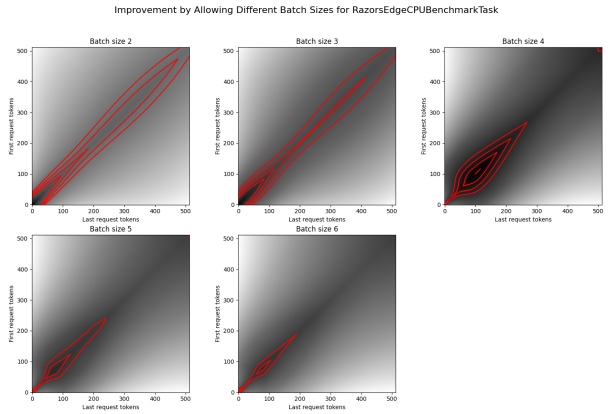


Figure 19: Improvement from allowing variable batch sizes (real-model CPU performance contours).

Notebook source: `demos/cpu/razors_edge_cpu_benchmark_graphs.ipynb`.



7.10 Summary of Findings

Across both workloads, we observe:

- Razor's Edge outperforms simple batching strategies
- Razor's Edge enables performance gains even when simple batching strategies degrade performance relative to no batching
- Input size dependent batching structure: batching efficiency exhibits a transition driven by memory-bound vs. compute-bound regimes.
- Throughput improvement under load: Razor's Edge increases effective throughput as concurrency grows by enabling better batch construction.

Together, these results demonstrate that, on the evaluated workloads and setup, Razor's Edge improves batching efficiency and exhibits load-adaptive behavior relative to the baselines considered in this paper.

8 Limitations and Future Work

- Partitioning and candidate evaluation add CPU overhead. This may cause slowdowns for very small models, CPU-constrained environments, or large batch sizes.
- Timing estimators could be created directly from model architecture and benchmarking basic operations.
- It is unknown why FIFO sometimes shows slightly higher throughput in certain settings. One possible explanation is that FIFO can randomly select more "efficient" small, fast batches, while tail-aware policies (for example MINMAX) may sometimes deprioritize those batches.

Future work includes:

1. Better estimators by analysis of model components
2. Estimator uncertainty modeling and online recalibration.
3. Throughput may be further improved by explicitly selecting the most hardware-efficient candidate batches.
4. Open-loop and bursty traffic evaluation with SLO-oriented metrics (for example throughput/latency frontier and p95/p99 miss-rate curves).
5. Broader baseline comparisons against additional production-oriented dynamic batching policies under matched hardware/software settings.

9 Conclusion

Razor's Edge provides a practical batching framework that unifies throughput optimization and latency objectives for variable-size inference workloads. Our core contribution is a systems synthesis: DP-based contiguous partitioning on sorted requests, production-oriented online strategy selection (`FIFO`, `MINMAX`, `GUARDED_BATCH_SIZE`), and practical estimator construction for deployment. In the evaluated synthetic and real (`BAAI/bge-m3`, `jinaai/jina-embeddings-v2-base-en`) settings with calibrated timing estimators, this combination improves throughput and maintains favorable latency behavior relative to the tested FIFO/fixed-cap baselines.

Validated in this paper:

- On the reported experiments, the proposed scheduler outperforms the defined baselines on throughput for the tested variable-size batched inference tasks.
- The observed contour structure and load-scaling behavior are reproducible within the tested synthetic and `BAAI/bge-m3`, `jinaai/jina-embeddings-v2-base-en` setups.

Not claimed:

- A universal guarantee across all serving stacks, models, and hardware.
- Global optimality of the multi-group online latency ordering pass.

Accordingly, the method should be interpreted as an objective-driven, empirically supported scheduler design for the evaluated workload class, rather than a general theorem about all inference systems.

This manuscript emphasizes transparent release of code, estimator artifacts, and reproducible replay analyses; expanded system-level evaluations and broader baseline suites are a natural next step.

10 Funding and Conflict of Interest

- **Funding:** None.
- **Conflict of interest:** The author declares no competing financial or non-financial interests related to this work.

11 Ethical Considerations / Potential Misuse

This work improves serving efficiency and resource allocation for batched inference systems. In deployment, operators should monitor service stability when these mechanisms are integrated. Efficiency gains can also vary based on random variance at startup or request size distribution. Latency objectives should be selected carefully based on the application. Startup time will increase due to estimator creation, which can degrade service in some applications.

12 Reproducibility Notes

Repository: https://github.com/arrmansa/Razors-Edge-batching-scheduler. To reproduce results (throughput, strategy comparisons, and historical RMS transitivity checks) and figures, install dependencies from `requirements.txt`, open each notebook in `demos/`, and execute **Run All** from the first cell in a clean kernel. Generated plots will be saved in `images/`.

Seeds are set in notebooks where appropriate for deterministic or near-deterministic sections of the experiments.

For the estimator-array replay validation and ablation in Section 7.2.1-7.2.2:

- generate estimator arrays with `demos/scheduler_tests/save_estimators_for_simulation.ipynb`
- run `python demos/scheduler_tests/generate_simulation_results.py`
- inspect `simulation results.md` and `demos/scheduler_tests/simulation_results.json`
- optional notebook walkthroughs:
  - `demos/scheduler_tests/simulation_strategy_tests_saturated_enhanced.ipynb`
  - `demos/scheduler_tests/simulation_strategy_tests_saturated_ablation.ipynb`

We keep benchmark-critical dependencies in `requirements.txt`; non-benchmark package version changes (for example notebook tooling/plotting helpers) should not materially change measured throughput trends.

Absolute throughput values can vary across hardware and runtime conditions, but the contour-shape phenomenon (”razor's edge” transition between efficient and inefficient batching regions) is expected to be present on most modern hardware and models.

All figures have source notebooks provided.

The paper text describes implemented behavior and references concrete function names in:

- `src/razors_edge/optimal_batching.py`
- `src/razors_edge/optimal_benchmarking.py`
- `src/razors_edge/razors_edge_compute_task.py`
- `src/executor/base_batched_compute_task.py`
- `src/executor/process_manager.py`

13 References

[1] W. E. Smith, ”Various optimizers for single-stage production,” *Naval Research Logistics Quarterly*, vol. 3, no. 1-2, pp. 59-66, 1956.

[2] U. Bagchi, Y. L. Chang, and R. S. Sullivan, ”Minimizing squared deviation of completion times about a common due date,” Manag. Sci., vol. 33, no. 7, pp. 894–906, Jul. 1987.

[3] S. K. Gupta and T. Sen, ”Minimizing a quadratic function of job lateness on a single machine,” Eng. Costs Prod. Econ., vol. 7, no. 3, pp. 187–194, Sep. 1983.

[4] NVIDIA, ”Dynamic Batcher,” Triton Inference Server Documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html (accessed 2026-03-24).

[5] mixedbread-ai, ”batched” (GitHub repository). https://github.com/mixedbread-ai/batched (accessed 2026-03-24).

[6] michaelfeil, ”infinity” integrations (GitHub repository). https://github.com/michaelfeil/infinity?tab=readme-ov-file#integrations (accessed 2026-03-24).

[7] Hugging Face, ”Pipeline batch inference” documentation. https://huggingface.co/docs/transformers/pipeline_tutorial#batch-inference (accessed 2026-03-24).